

Fundamentos Matemáticos de la Convolución Aperiódica basada en Teselación de Penrose para Redes Neuronales Ligeras

Axel Eduardo Urbina Secundino

<https://github.com/AEUS-06> | axedus06@gmail.com

Preprint trabajo exploratorio

12/04/2026

Resumen

Las redes neuronales convolucionales clásicas operan sobre cuadrículas regulares, lo que les impone, casi sin que nos demos cuenta, una preferencia hacia patrones geométricos alineados con ejes ortogonales. En este trabajo se explora una alternativa: usar la *teselación de Penrose P2* (cometas y dardos) como soporte geométrico del kernel convolucional. Esta partición del plano es aperiódica y presenta simetría local de orden 5, con una estructura completamente gobernada por la razón áurea $\varphi = \frac{1+\sqrt{5}}{2}$ y la sucesión de Fibonacci. Derivamos la formulación matemática del *kernel de Penrose*, lo reinterpretemos como una convolución sobre grafos, describimos el algoritmo de subdivisión (deflación) implementado en C, y proponemos un esquema de compresión de pesos que aprovecha la autosimilitud fractal de la teselación. El aporte es principalmente teórico-exploratorio; el código está disponible bajo licencia GNU LGPL v3.

1. Introducción

Hay algo que me llamó la atención mientras estudiaba las CNN estándar: todos los kernels viven en una cuadrícula cuadrada. Eso suena natural, pero tiene un costo poco discutido. Al usar retículas regulares con separación uniforme entre puntos, el modelo termina favoreciendo patrones a 0° y 90° , y su campo receptivo crece siempre de manera cuadrada. Dos restricciones que nadie pide explícitamente, pero que están ahí desde el primer `Conv2d`.

Los modelos ligeros modernos, como MobileNet [7] o SqueezeNet [8], atacan el problema desde el ángulo de la eficiencia computacional: factorizan las operaciones para reducir parámetros y multiplicaciones. Lo que *no* hacen es cuestionar la forma del kernel. Siguen siendo cuadrados.

Una pregunta que surge casi sola es: ¿qué pasa si el conjunto de puntos de muestreo deja de ser periódico? Los quasi-cristales descubiertos por Shechtman et al. [5] muestran que la materia puede organizarse con simetría local de orden 5 sin ser periódica en sentido traslacional. La analogía con los kernels de redes neuronales es directa.

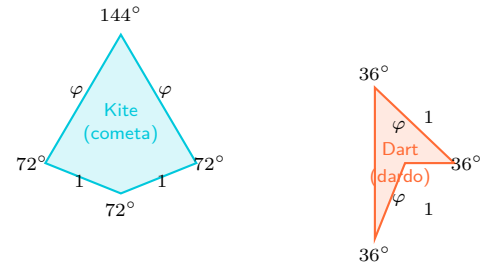


Figura 1: Las dos piezas básicas de la teselación de Penrose P2. Los lados tienen longitudes 1 y $\varphi \approx 1,618$.

La teselación de Penrose P2 [2] ofrece exactamente esa estructura geométrica: cubre el plano completo con dos tipos de cuadriláteros (cometa y dardo), sin repetirse periódicamente, pero con una organización interna dictada por φ y Fibonacci. Este trabajo explora si esa geometría puede reemplazar la cuadrícula cuadrada como base del kernel convolucional, y qué propiedades matemáticas y computacionales surgen de esa decisión. El prototipo implementado se llama *TeselCore* y está escrito en C.

2. Fundamento Matemático

2.1. La razón áurea y Fibonacci

Todo empieza con un número. Uno solo.

Definición 1 (Razón áurea). Sea $\varphi \in \mathbb{R}_{>0}$ la raíz positiva de la ecuación $\varphi^2 = \varphi + 1$, cuya forma cerrada es $\varphi = \frac{1+\sqrt{5}}{2} \approx 1,6180 \dots$

Lo que hace especial a φ en este contexto es su propiedad recíproca: $\frac{1}{\varphi} = \varphi - 1$. Eso significa que los lados de las tejas de Penrose, que miden 1 y φ (Fig. 1), no son valores arbitrarios: surgen de la misma ecuación cuadrática. La sucesión de Fibonacci $F_n = F_{n-1} + F_{n-2}$ también aparece de manera natural: su cociente F_{n+1}/F_n converge a φ , y la proporción entre cometas y dardos en cualquier teselación grande tiende a $\varphi^2 \approx 2,618$.

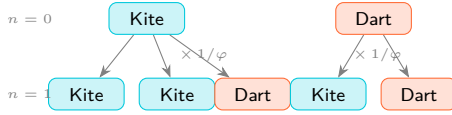


Figura 2: Árbol de deflación: cómo se subdivide cada teja según las reglas (2)–(3).

2.2. Teselación de Penrose P2

Definición 2 (Teselación P2 [2, 3]). *La teselación de Penrose P2 recubre el plano usando únicamente los dos cuadriláteros de la Figura 1. Es aperiódica: ningún vector de traslación la transforma en sí misma, pero internamente muestra simetría local de orden 5 en cualquier región acotada.*

Los vértices de cada teja se ubican sobre rayos angulares de la forma $\theta_k = \frac{k\pi}{5}$, con $k \in \mathbb{Z}$. Para un kite de escala s , las coordenadas son:

$$\mathbf{p}_k = s(\cos \theta_k, \sin \theta_k), \quad \mathbf{p}'_k = \frac{s}{\varphi} \left(\cos(\theta_k + \frac{\pi}{5}), \sin(\theta_k + \frac{\pi}{5}) \right). \quad (1)$$

2.3. Algoritmo de deflación

Una propiedad clave de Penrose P2 es que se puede “acercar” o “alejar”: cada teja se puede subdividir en tejas más pequeñas del mismo tipo. A eso se le llama *deflación*.

Proposición 1 ([2]). *Cada teja se subdivide escalándola por el factor $1/\varphi$:*

$$\text{Kite} \rightarrow 2 \text{Kite} + 1 \text{Dart}, \quad (2)$$

$$\text{Dart} \rightarrow 1 \text{Kite} + 1 \text{Dart}. \quad (3)$$

Los nuevos vértices se obtienen usando la razón áurea:

$$Q = C + (P - C)/\varphi, \quad (4)$$

$$R = L + (C - L)/\varphi. \quad (5)$$

Cada vez que se aplica la deflación, el número total de tejas crece aproximadamente como $T_n \sim \varphi^{2n} \cdot 10$ (Tabla 1).

Corolario 1. *La razón T_n^K/T_n^D (cometas sobre dardos) tiende a φ conforme n crece, siguiendo el ritmo de la sucesión de Fibonacci.*

Cuadro 1: Crecimiento de tejas por nivel de deflación.

Nivel n	T_n	T_n^K/T_n^D	$\varphi^{2n} \cdot 10$
0	10	1,000	10,0
1	20	1,500	26,2
2	60	1,571	68,5
3	160	1,600	179,4
4	420	1,615	469,9

3. Representación como Grafo

3.1. El grafo de Penrose

Llegado a este punto, resulta conveniente dejar de ver la teselación como una figura geométrica y empezar a pensarla como un grafo. Al nivel n se define $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ donde:

- $\mathcal{V}$: cada teja es un nodo, $|\mathcal{V}| = T_n$.
- $\mathcal{E}$: dos tejas comparten arista si y sólo si comparten un lado completo.

Un detalle importante: el grado de cada nodo varía entre 3 y 7 según el tipo y la posición de la teja. Eso hace que este grafo sea fundamentalmente distinto a los grafos que subyacen a una CNN con retícula cuadrada, donde todo nodo interior tiene exactamente 4 vecinos.

3.2. Construcción de la tabla de vecindad

En el prototipo, la vecindad se calcula ordenando las aristas de todas las tejas y agrupando las que coinciden en ambos extremos:

Listing 1: Vecindad por aristas compartidas.

```

1 for (int i = 0; i < T; i++) {
2   int* vs = actual.buf[i].v;
3   for (int p = 0; p < 4; p++) {
4     int u = vs[p], v = vs[(p+1)%4];
5     int a = u < v ? u : v;
6     int b = u < v ? v : u;
7     edges[ecount++] = (_Edge){a,b,i};
8   }
9 }
10 qsort(edges, ecount,
11        sizeof(_Edge), _cmp_edge);
12 for (int i = 0; i < ecount; ) {
13   int j = i+1;
14   while (j < ecount &&
15          edges[j].a==edges[i].a &&
16          edges[j].b==edges[i].b) j++;
17   /* tiles en [i..j-1]: son vecinos */
18   for (int p=i; p<j; p++)
19     for (int q=p+1; q<j; q++) {
20       int t1=edges[p].tile;
21       int t2=edges[q].tile;
22       /* agregar t2<->t1 */
23     }
24   i = j;
25 }
```

3.3. Conexión con GNN

Una vez que tenemos el grafo, podemos instanciar sobre él cualquier convolución en grafo conocida. La formulación de Kipf y Welling [9]:

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(l)} W^{(l)}\right) \quad (6)$$

se aplica directamente sobre $\mathcal{G}$, con la ventaja de que la topología del grafo no es algo que el modelo aprende: viene fijada por la geometría de la teselación. Eso ahorra parámetros y hace que la estructura sea interpretable.

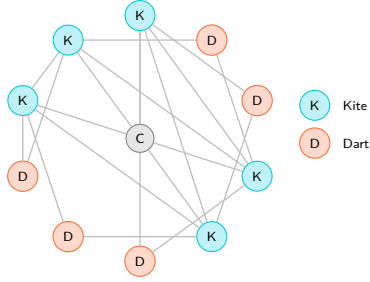


Figura 3: Grafo $\mathcal{G}$ en nivel 0 (10 tejas, semilla “sol”). El grado de cada nodo varía entre 3 y 7.

4. El Kernel de Penrose

4.1. Formulación imagen

Dado un tensor de entrada $\mathbf{x} \in \mathbb{R}^{H \times W \times C_{in}}$, a cada teja t se le asigna un centro normalizado $(n_x^t, n_y^t) \in [0, 1]^2$. Con paso espacial s , el punto donde se muestrea la imagen es:

$$\mathbf{f}_t(i, j) = (js + (n_x^t - \frac{1}{2})s, is + (n_y^t - \frac{1}{2})s). \quad (7)$$

Usando interpolación bilineal $\mathcal{B}$ para recuperar valores en posiciones no enteras, la convolución de Penrose queda:

$$y_{c_{out}}(i, j) = b_{c_{out}} + \sum_{t=1}^{T_n} \sum_{c=1}^{C_{in}} w_{c_{out}, c, t} \cdot \mathcal{B}(\mathbf{x}[\cdot, \cdot, c], \mathbf{f}_t(i, j)) \quad (8)$$

4.2. Formulación en grafo

Cuando el input ya está en forma de grafo, $\mathbf{X} \in \mathbb{R}^{B \times C_{in} \times T_n}$, la operación se reescribe como:

$$Y_{b, c_{out}, i} = b_{c_{out}} + \sum_{c=1}^{C_{in}} \sum_{k=0}^{K-1} w_{c_{out}, c, k} \cdot X_{b, c, \nu_k(i)}, \quad (9)$$

donde $\nu_0(i) = i$ (la teja misma) y $\nu_k(i)$ referencia al k -ésimo vecino de i en el grafo $\mathcal{G}$.

Listing 2: Núcleo de `tc_conv_penrose_grafo`.

```

1 for (int b = 0; b < B; b++)
2 for (int co = 0; co < Cout; co++)
3 for (int i = 0; i < N; i++) {
4     float suma = db ? db[co] : 0.0f;
5     const tc_teja_penrose* tj =
6         &teselacion->tejas[i];
7     for (int ci = 0; ci < Cin; ci++) {
8         suma += dk[co*Cin*K+ci*K+0]
9             * de[b*Cin*N+ci*N+i];
10        for (int nv = 0;
11             nv < tj->num_vecinos
12             && (nv+1) < K; nv++) {
13            int v = tj->vecinos[nv];
14            suma += dk[co*Cin*K+ci*K+(nv+1)]
15                * de[b*Cin*N+ci*N+v];
16        }
17    }
18    ds[b*Cout*N+co*N+i] = suma;
19 }
```

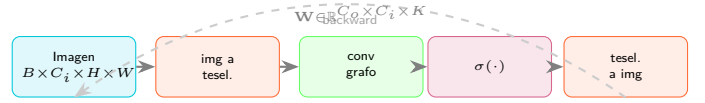


Figura 4: Pipeline de *TeselCore*: la imagen se proyecta a la teselación, se aplica la conv. en grafo con activación, y se regresa al espacio imagen. La flecha punteada indica el paso de retropropagación.

4.3. Inicialización He adaptada

Dado que el número de conexiones por nodo varía, se adapta la inicialización de He [11] al tamaño total de la teselación:

$$\sigma_w = \sqrt{\frac{2}{C_{in} \cdot T_n}}, \quad w \sim \mathcal{N}(0, \sigma_w^2). \quad (10)$$

Observación 1. La ec. (8) contiene a la convolución estándar como caso especial: si $T_n = K^2$ y los centros de las tejas forman una cuadrícula uniforme, la expresión se reduce exactamente a una convolución cuadrada ordinaria.

5. Implementación: Aspectos Clave

5.1. Corrección del bug de geometría degenerada

Durante el desarrollo apareció un error que, en retrospectiva, era difícil de detectar con sólo mirar la salida numérica. La primera versión del algoritmo calculaba correctamente los índices i_L e i_R de los vértices laterales, pero después los descartaba silenciosamente y duplicaba el vértice i_P :

Listing 3: Bug: vértice i_P duplicado.

```

(void)iL; (void)iR; /* BUG: se descartan */
_bb_agregar(&actual, tipo, i0, iC, iP, iP, ang);
```

Listing 4: =origen, v[1]=izq, v[2]=punta, v[3]=der.]Fix: convención v[0]=origen, v[1]=izq, v[2]=punta, v[3]=der.

```

_bb_agregar(&actual, tipo, i0, iL, iP, iR, ang);
```

El efecto práctico era que todas las tejas del nivel 0 eran triángulos degenerados disfrazados de cuadriláteros. El error se propagaba a todos los niveles superiores de deflación, generando una teselación geométricamente incorrecta donde las activaciones y los pesos no tenían ningún significado espacial real. Tras la corrección, la simetría de orden 5 aparece visualmente de manera limpia en todos los niveles.

5.2. Proyección imagen / teselación

La función `tc_imagen_a_teselacion` toma los centros normalizados de cada teja y muestrea la imagen en esas coordenadas usando interpolación bilineal. La operación inversa, `tc_teselacion_a_imagen`, redistribuye los valores con *splatting* bilineal ponderado y normaliza al final para evitar artefactos en zonas de alta superposición:

Listing 5: Normalización del splatting.

```
for (size_t i = 0; i < total; i++)
    if (dp[i] > 1e-8f) ds[i] /= dp[i];
```

6. Compresión Estructural Adaptativa

Una consecuencia interesante de la deflación es que la teselación de nivel n *contiene* (en sentido geométrico) copias a escala de la teselación de nivel $n - 1$. Eso recuerda a los sistemas de funciones iteradas de Barnsley [12].

Si esa auto-similitud existe en la geometría, ¿existe también en los pesos aprendidos? La hipótesis es que sí, al menos parcialmente. Bajo esa suposición, se puede aproximar el tensor de pesos del nivel n a partir del del nivel $n - 1$:

$$\mathbf{W}^{(n)} \approx \mathcal{F}(\mathbf{W}^{(n-1)}) + \Delta \mathbf{W}, \quad (11)$$

donde $\mathcal{F}$ es una transformación contráctil que el modelo puede aprender, y $\Delta \mathbf{W}$ captura la diferencia residual. En principio, almacenar $\mathcal{F}$ y $\Delta \mathbf{W}$ en lugar de $\mathbf{W}^{(n)}$ entero podría reducir la memoria por un factor cercano a $\varphi^2 \approx 2.6$.

Para saber qué pesos comprimir sin comprometer el rendimiento, se puede usar una puntuación de importancia basada en la diagonal de la información de Fisher [14]:

$$\mathcal{I}_{c_{out}, c, t} = \mathbb{E} \left[\left(\frac{\partial \mathcal{L}}{\partial w_{c_{out}, c, t}} \right)^2 \right]. \quad (12)$$

Los pesos con $\mathcal{I} < \tau$ son candidatos a comprimirse fractalmente; los de alta importancia se conservan en precisión completa. Esta combinación da lugar a un esquema de compresión que se adapta a lo que el modelo realmente usa.

7. Ventajas, Limitaciones y Trabajo Futuro

Cobertura angular más rica. Los puntos de muestreo de la teselación cubren 0° , 72° , 144° , 216° y 288° de forma natural, frente a los dos únicos ejes de la cuadrícula cuadrada. Para objetos con simetría pentagonal —manos, ciertas flores, algunos cristales— esa cobertura adicional podría reducir el sesgo de orientación sin aumentar el número de parámetros.

Aperiodicidad como regularización implícita. La falta de periodicidad traslacional dificulta que el kernel aprenda a simplemente memorizar texturas que se repiten. Es una forma de regularización estructural que no requiere ningún término extra en la función de pérdida.

Multi-escala sin submuestreo. Cada nivel de deflación opera a una escala diferente del espacio, lo cual es análogo a un esquema multi-resolución pero sin requerir operaciones de *pooling* explícitas.

Limitaciones actuales. El costo computacional del forward de la ec. (8) es $\mathcal{O}(B \cdot C_{out} \cdot C_{in} \cdot T_n \cdot H_s \cdot W_s)$, un factor T_n/K^2 mayor que el de una convolución cuadrada de kernel K . Además, a la fecha no se ha implementado el backward en modo imagen completo ni se han realizado experimentos cuantitativos sobre ningún benchmark estándar. Por lo tanto, las ventajas descritas son por ahora teóricas.

Líneas de trabajo futuro:

1. Implementar el backward completo en modo imagen.
2. Evaluar en MNIST, CIFAR-10 y conjuntos de datos de reconocimiento de gestos o señas.
3. Integrar el esquema de compresión fractal (11).
4. Explorar una extensión tridimensional inspirada en los quasi-cristales icosaédricos.
5. Comparar sistemáticamente con convoluciones deformables [16].

8. Conclusión

Este trabajo presenta los cimientos matemáticos de una convolución aperiódica basada en la teselación de Penrose P2. La ec. (8) generaliza la convolución estándar al desvincular los puntos de muestreo de una cuadrícula regular, y la ec. (9) extiende esa idea al dominio de grafos, donde la conectividad variable entre tejas es una característica, no un problema. La auto-similitud deflacionaria de la teselación abre una posible vía hacia esquemas de compresión estructurada de modelos, aunque validar esa hipótesis requiere experimentación que queda como trabajo futuro inmediato.

El código fuente del prototipo está disponible en el repositorio TeselCore bajo GNU LGPL v3.

Todo el código que implementa las ecuaciones de este artículo es abierto y puede descargarse para su reproducción independiente.

Referencias

- [1] R. Penrose, “The role of aesthetics in pure and applied mathematical research,” *Bull. Inst. Math. Appl.*, vol. 10, pp. 266–271, 1974.
- [2] R. Penrose, “Pentaplexity: A class of non-periodic tilings of the plane,” *Eureka*, vol. 39, pp. 16–22, 1979.

- [3] B. Grünbaum y G. C. Shephard, *Tilings and Patterns*. W. H. Freeman, New York, 1987.
- [4] M. Gardner, “Extraordinary nonperiodic tiling that enriches the theory of tiles,” *Scientific American*, vol. 236, no. 1, pp. 110–121, 1977.
- [5] D. Shechtman et al., “Metallic phase with long-range orientational order and no translational symmetry,” *Phys. Rev. Lett.*, vol. 53, pp. 1951–1953, 1984.
- [6] Y. LeCun et al., “Gradient-based learning applied to document recognition,” *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [7] A. G. Howard et al., “MobileNets: Efficient convolutional neural networks for mobile vision applications,” *arXiv:1704.04861*, 2017.
- [8] F. N. Iandola et al., “SqueezeNet: AlexNet-level accuracy with 50x fewer parameters,” *arXiv:1602.07360*, 2016.
- [9] T. N. Kipf y M. Welling, “Semi-supervised classification with graph convolutional networks,” *Proc. ICLR*, 2017.
- [10] P. Veličković et al., “Graph attention networks,” *Proc. ICLR*, 2018.
- [11] K. He et al., “Delving deep into rectifiers,” *Proc. ICCV*, pp. 1026–1034, 2015.
- [12] M. F. Barnsley, *Fractals Everywhere*. Academic Press, 1988.
- [13] A. E. Jacquin, “Image coding based on a fractal theory of iterated contractive image transformations,” *IEEE Trans. Image Process.*, vol. 1, pp. 18–30, 1992.
- [14] J. Kirkpatrick et al., “Overcoming catastrophic forgetting in neural networks,” *PNAS*, vol. 114, pp. 3521–3526, 2017.
- [15] S. Han et al., “Learning both weights and connections for efficient neural networks,” *NeurIPS*, pp. 1135–1143, 2015.
- [16] J. Dai et al., “Deformable convolutional networks,” *Proc. ICCV*, pp. 764–773, 2017.